

# Pixelium Consent Commerce

---

## Document d'architecture — Stage Pixelium 2026

Projet : Commerce agentique avec consentement utilisateur (A2A + mandats AP2-inspired)

Auteur : Stagiaire · Destinataire : Encadrant

Date : Juillet 2026

**Résumé exécutif :** Nous avons développé un prototype de boutique en ligne où des agents IA peuvent rechercher des produits et effectuer des achats *uniquement* après validation explicite de l'utilisateur. Chaque transaction suit une chaîne de mandats signés : **Intent** → **Cart** → **Payment**. Un broker central valide tout avant qu'un agent de paiement ne débite (simulation). Un assistant RAG (embeddings + vector store + LLM) aide l'utilisateur à choisir quoi acheter.

## Table des matières

1. Contexte et objectifs
2. Architecture globale du système
3. Structure du monorepo
4. Chaîne de mandats (consentement)
5. Flux utilisateur (realtime et delegated)
6. Consent Broker (orchestrateur)
7. Dashboard React (interface utilisateur)
8. Architecture RAG (assistant shopping)
9. Agent E-commerce (Product Agent)
10. Agent Paiement (Payment Agent)
11. Relation RAG ↔ Agents
12. Technologies utilisées
13. Sécurité et tests
14. Limites et perspectives
15. Guide de démonstration

# 1. Contexte et objectifs

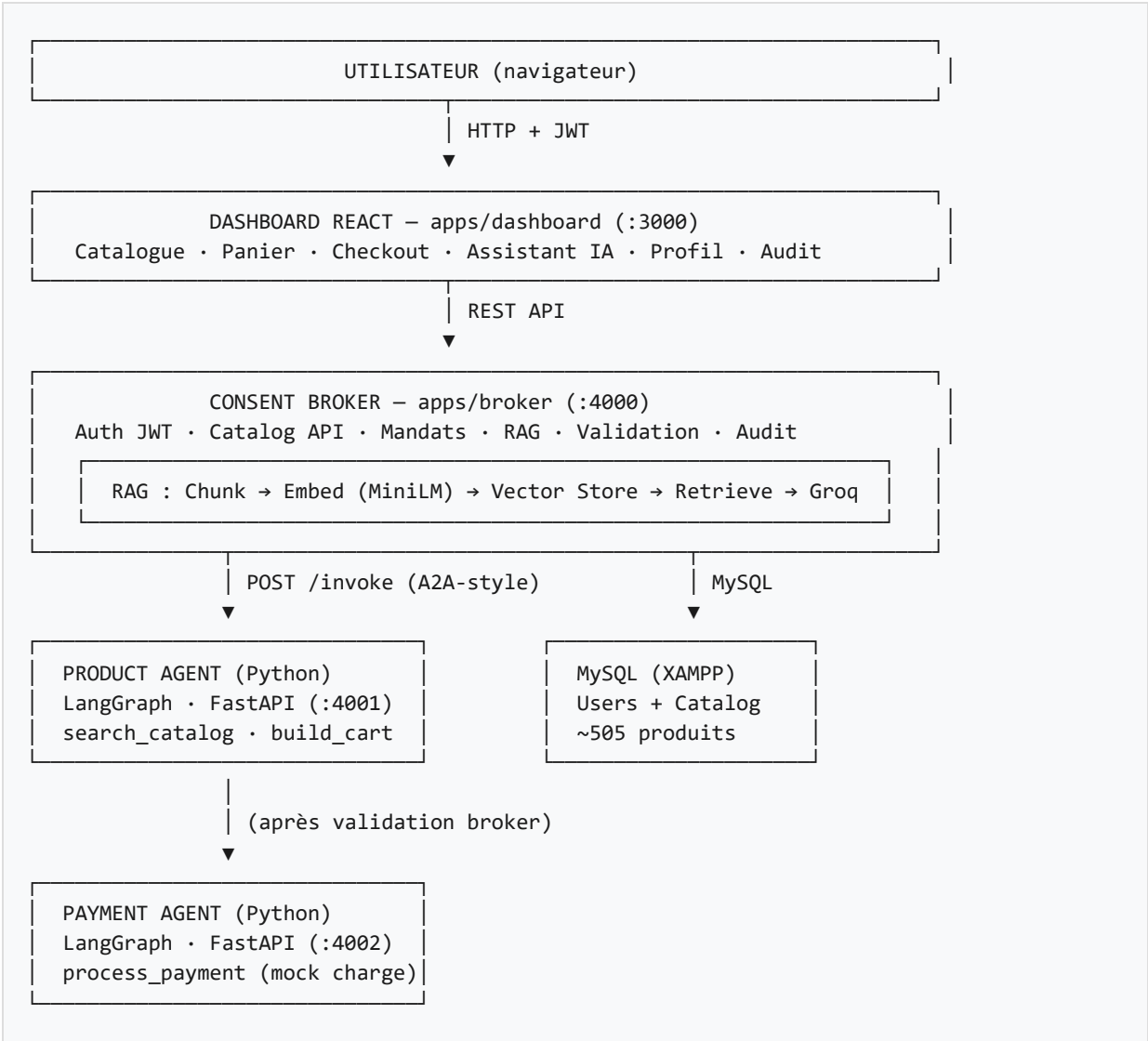
## 1.1 Problématique

Les assistants IA peuvent aujourd'hui rechercher des produits, remplir un panier et parfois initier un paiement sans contrôle clair pour l'utilisateur. Notre projet démontre qu'il est possible d'automatiser l'achat tout en conservant le **consentement humain** à chaque étape critique.

## 1.2 Objectifs du prototype

- Implémenter une chaîne de mandats inspirée du protocole **AP2** (Agent Payments Protocol, Google).
- Faire communiquer deux agents via le protocole **A2A** (Agent-to-Agent).
- Centraliser la validation dans un **Consent Broker**.
- Proposer un assistant IA avec **RAG** pour recommander des produits du catalogue.
- Journaliser toutes les opérations dans une piste d'**audit**.

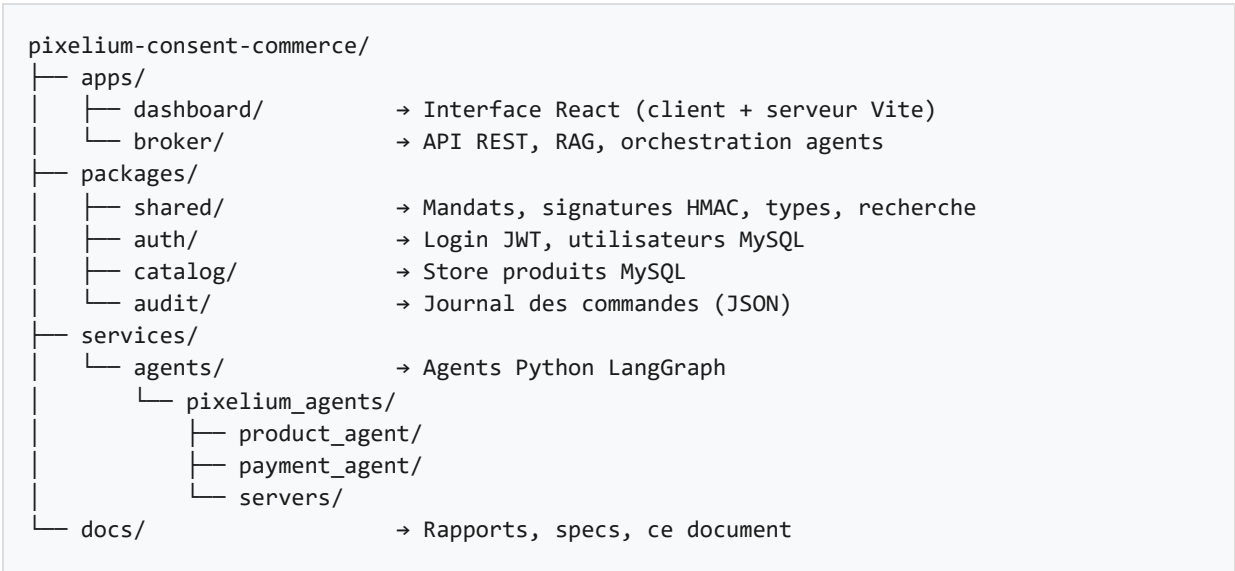
## 2. Architecture globale du système



**Principe clé :** le navigateur ne communique *jamais* directement avec les agents Python. Tout passe par le broker, seul point de contrôle.

| Composant      | Port | Technologie                  | Rôle                                  |
|----------------|------|------------------------------|---------------------------------------|
| Dashboard      | 3000 | React 19 + Vite              | Interface boutique et consentement    |
| Consent Broker | 4000 | Node.js + Express            | Orchestration, RAG, validation, audit |
| Product Agent  | 4001 | Python + LangGraph + FastAPI | Recherche catalogue + Cart Mandate    |
| Payment Agent  | 4002 | Python + LangGraph + FastAPI | Preuve mandats + paiement simulé      |
| MySQL          | 3306 | XAMPP                        | Utilisateurs et catalogue produits    |

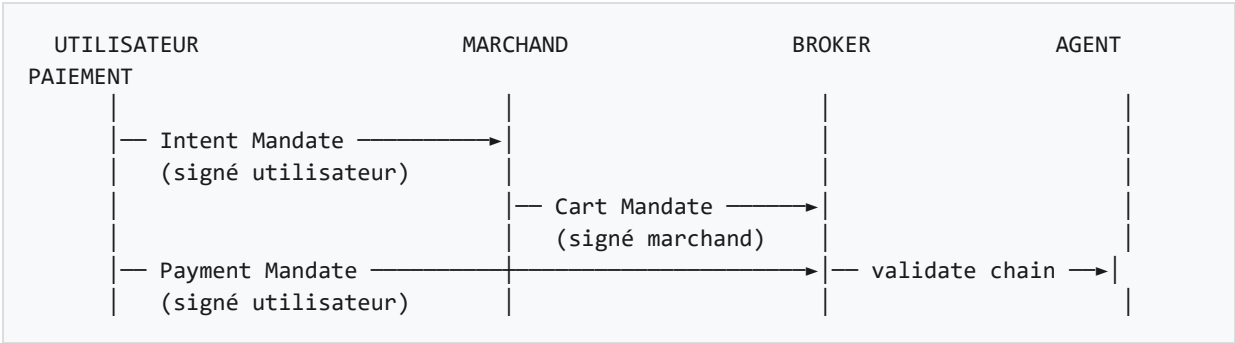
### 3. Structure du monorepo



Commande unique pour lancer les 4 services : `npm run dev`

### 4. Chaîne de mandats (consentement)

Inspirée du protocole AP2, chaque achat suit trois mandats signés et liés :



## 4.1 Structure d'un mandat

```
{
  "id": "uuid",
  "type": "intent | cart | payment",
  "version": "1.0",
  "createdAt": "ISO-8601",
  "expiresAt": "ISO-8601",
  "payload": { ... },
  "signerId": "user | merchant | ...",
  "signature": "HMAC-SHA256 hex",
  "parentMandateId": "lien vers mandat parent"
}
```

## 4.2 Signature

Algorithme : **HMAC-SHA256** sur la chaîne canonique :

```
type | signerId | parentMandateId | JSON payload (clés triées)
```

## 4.3 Contenu des mandats

| Mandat  | Signataire  | Contenu principal                                                           |
|---------|-------------|-----------------------------------------------------------------------------|
| Intent  | Utilisateur | Intention NL, budget max, SKUs autorisés, mode (realtime/delegated), expiry |
| Cart    | Marchand    | Items, sous-total, taxe (~8%), total, lien intentMandateId                  |
| Payment | Utilisateur | Montant (= total panier), liens cart + intent                               |

## 5. Flux utilisateur

### 5.1 Mode Realtime (utilisateur présent)

1. L'utilisateur ajoute des produits au panier ou utilise l'IA.
2. Checkout → le broker crée un Intent Mandate.
3. Le broker appelle le Product Agent → Cart Mandate.
4. L'UI affiche la revue de consentement (Review & pay).
5. L'utilisateur approuve → Payment Mandate → broker valide → Payment Agent charge (mock).

## **5.2 Mode Delegated (utilisateur absent)**

1. L'utilisateur signe à l'avance un Intent Mandate avec conditions (budget, SKUs, dates).
2. Un moniteur background (`delegated-monitor.ts`) surveille les conditions.
3. Quand les conditions sont remplies, le broker exécute automatiquement cart + payment.

## 6. Consent Broker (orchestrateur)

**Fichier principal :** apps/broker/src/server.ts · broker.ts

### 6.1 Responsabilités

- API REST authentifiée (JWT)
- Exposition du catalogue MySQL
- Création et validation des mandats
- Appel des agents Python via POST /invoke
- Pipeline RAG pour l'assistant shopping
- Journal d'audit (packages/audit)

### 6.2 Endpoints principaux

| Endpoint                   | Rôle                                       |
|----------------------------|--------------------------------------------|
| POST /api/auth/login       | Authentification JWT                       |
| GET /api/catalog           | Liste produits paginée                     |
| POST /api/checkout/prepare | Prépare Intent + Cart mandats              |
| POST /api/submit           | Soumet la chaîne complète au Payment Agent |
| POST /api/ai/advise        | Assistant RAG (recommandations)            |
| POST /api/ai/prepare       | IA → intent + cart pour checkout           |
| GET /api/audit/orders      | Historique commandes                       |
| GET /health                | État services + index RAG                  |

## 7. Dashboard React (interface utilisateur)

**Chemin :** apps/dashboard/src/client/

### 7.1 Fonctionnalités

- Parcours catalogue avec catégories et pagination
- Panier et checkout avec modale de consentement

- Panel assistant IA (4 actions)
- Profil utilisateur et historique commandes
- Dashboard audit (admin)

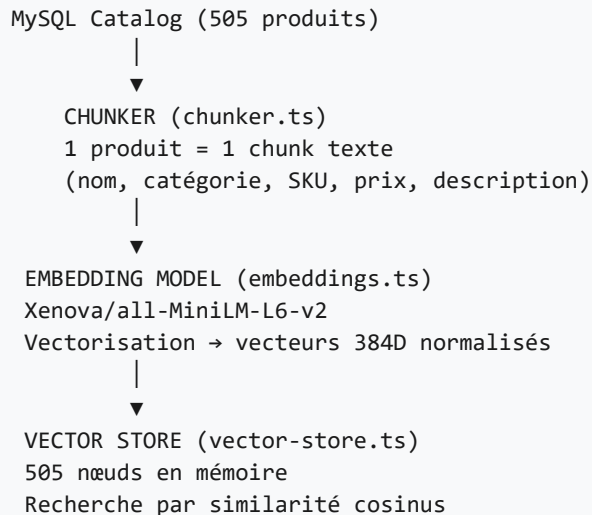
7.2 Panel assistant IA

| Bouton          | Action                                | Appelle les agents ? |
|-----------------|---------------------------------------|----------------------|
| Get advice      | RAG — recommandations                 | Non                  |
| Search catalog  | Recherche mots-clés                   | Non                  |
| Parse intent    | Comprendre l'intention d'achat (Groq) | Non                  |
| Search & review | Prépare checkout avec mandats         | Oui (Product Agent)  |

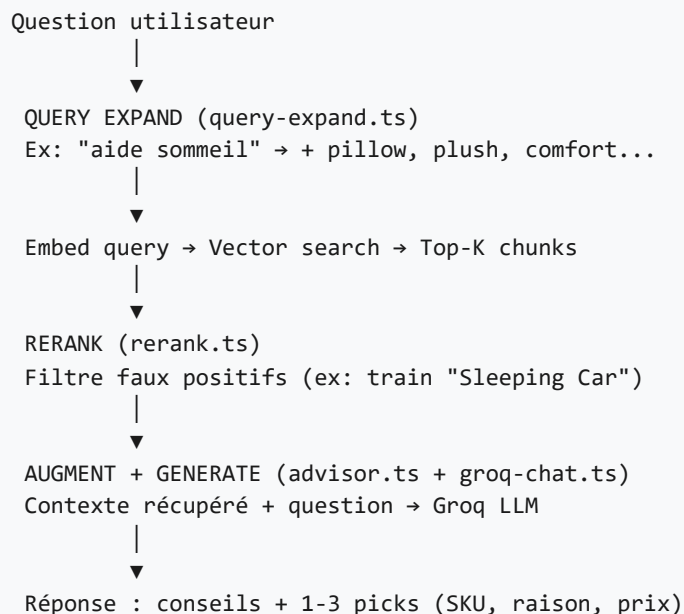
## 8. Architecture RAG (assistant shopping personnel)

Le RAG vit **dans le broker** (apps/broker/src/rag/). Il aide l'utilisateur à *décider* quoi acheter, sans créer de mandat ni payer.

### 8.1 Pipeline d'indexation (au démarrage du broker)



### 8.2 Pipeline de requête (runtime)



### 8.3 Modules RAG

| Fichier         | Rôle dans l'architecture RAG standard        |
|-----------------|----------------------------------------------|
| chunker.ts      | Chunking (documents → segments)              |
| embeddings.ts   | Embedding model (vectorisation)              |
| vector-store.ts | Vector store (index + recherche)             |
| indexer.ts      | Pipeline d'indexation complet                |
| retriever.ts    | Retrieve (query → embed → search)            |
| query-expand.ts | Enrichissement sémantique de la requête      |
| rerank.ts       | Re-classement intent-aware                   |
| advisor.ts      | Augment + Generate (prompt + LLM)            |
| groq-chat.ts    | Client Groq résilient (SDK + fetch fallback) |

## 9. Agent E-commerce (Product Agent — :4001)

**Technologie :** Python · FastAPI · LangGraph

**Chemin :** services/agents/pixelium\_agents/product\_agent/

### 9.1 Graph LangGraph



### 9.2 Compétences (skills A2A)

| Skill          | Action     | Entrée                  | Sortie                |
|----------------|------------|-------------------------|-----------------------|
| search_catalog | search     | query                   | products[], rankedSku |
| build_cart     | build_cart | intentMandate + items[] | cartMandate signé     |

### 9.3 Sous-agents parallèles (search)

- **sub\_search** — recherche mots-clés dans le catalogue
- **sub\_filter** — garde uniquement les produits en stock
- **sub\_rank** — score heuristique (nom, catégorie)

### 9.4 Cart Builder

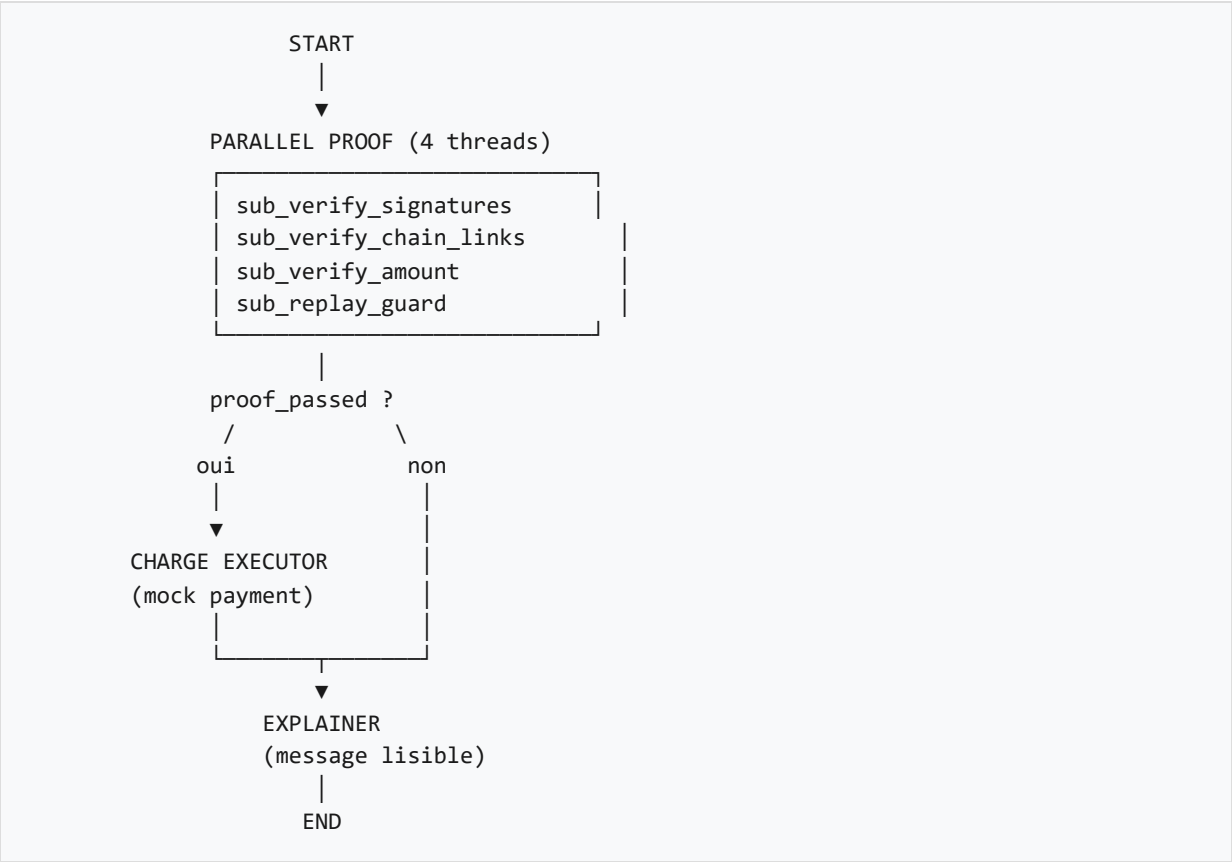
Valide les SKUs, calcule sous-total + taxe, crée un Cart Mandate signé HMAC par le marchand (signerId: merchant), lié au Intent Mandate parent.

## 10. Agent Paiement (Payment Agent — :4002)

**Technologie :** Python · FastAPI · LangGraph

**Chemin :** services/agents/pixelium\_agents/payment\_agent/

### 10.1 Graph LangGraph



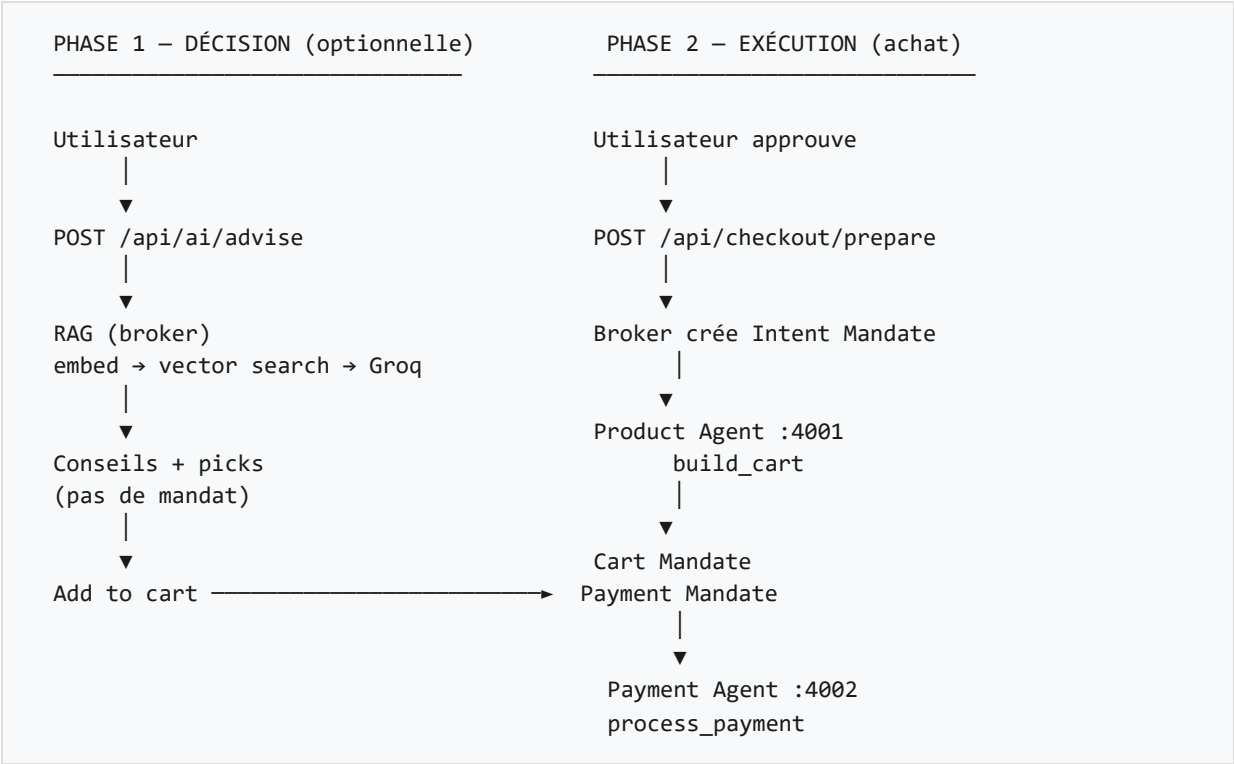
### 10.2 Vérifications de sécurité (100% déterministes)

| Vérification         | Bloque si...                                                  |
|----------------------|---------------------------------------------------------------|
| Signatures HMAC      | Signature invalide ou mandat expiré                           |
| Chaîne parent-enfant | Liens intent/cart/payment incohérents                         |
| Montants             | Paiement ≠ total panier, dépassement budget, SKU non autorisé |
| Replay guard         | Même paymentId déjà traité                                    |

### **10.3 Charge simulée**

Si toutes les preuves passent, génère un `transactionId` mock (ex: `txn_a1b2c3d4`). Aucun vrai paiement bancaire.

# 11. Relation RAG ↔ Agents



|               | RAG                 | Product Agent              | Payment Agent            |
|---------------|---------------------|----------------------------|--------------------------|
| Langage       | TypeScript (broker) | Python                     | Python                   |
| But           | Recommander         | Construire panier officiel | Valider et payer         |
| LLM           | Groq (conseils)     | Non (heuristique rank)     | Non (explainer template) |
| Consentement  | Non                 | Oui (Cart Mandate)         | Oui (Payment Mandate)    |
| Communication | Interne broker      | POST /invoke :4001         | POST /invoke :4002       |

**Le RAG ne parle jamais directement aux agents.** Le broker fait le lien entre découverte (RAG) et exécution (agents).

## 12. Technologies utilisées

| Couche              | Technologies                                          |
|---------------------|-------------------------------------------------------|
| Frontend            | React 19, TypeScript, Vite, CSS custom                |
| API / Orchestration | Node.js 22, Express 5, TypeScript                     |
| Agents              | Python 3.14, FastAPI, LangGraph, Pydantic             |
| Base de données     | MySQL (XAMPP) — users + catalog                       |
| IA / RAG            | Groq (Llama 3.3), @xenova/transformers (MiniLM-L6-v2) |
| Auth                | JWT + bcrypt                                          |
| Protocoles          | A2A (Agent Cards, /invoke), AP2-inspired mandates     |
| Audit               | JSON file store (packages/audit)                      |

## 13. Sécurité et tests

### 13.1 Mécanismes de protection

- Signatures HMAC sur chaque mandat
- Validation de chaîne par le broker avant envoi au Payment Agent
- Plafond de budget (maxPriceCents) dans Intent Mandate
- Liste blanche SKU optionnelle (mode delegated)
- Expiration temporelle des mandats
- Protection replay (double paiement bloqué)
- Authentification JWT sur toutes les routes sensibles

### 13.2 Tests

| Suite            | Commande                               | Couverture                                  |
|------------------|----------------------------------------|---------------------------------------------|
| Agents<br>Python | npm run test:agents                    | 20 tests unitaires LangGraph                |
| Adversariaux     | npm run test:adversarial               | Sig forgée, montant modifié, budget, replay |
| Demos E2E        | npm run demo:realtime / delegated / ai | Flux complets automatisés                   |
| Vérification     | npm run verify                         | Build + demos + tests                       |

### 14. Limites et perspectives

| Limite actuelle           | Perspective production                   |
|---------------------------|------------------------------------------|
| Paielements simulés       | Intégrer Stripe / PayPal réel            |
| Clés HMAC de démo         | ECDSA + clés device-bound                |
| Mandats simplifiés        | W3C Verifiable Credentials (AP2 complet) |
| Vector store en RAM       | Pinecone, Milvus, ou pgvector            |
| Jobs delegated en mémoire | Persistance Redis / DB                   |
| Catalogue jouets/hobbies  | Catalogue métier réel                    |

## 15. Guide de démonstration

### 15.1 Démarrage

```
cd pixelium-consent-commerce
npm install
pip install -r services/agents/requirements.txt
npm run dev
```

### 15.2 URLs

| URL                          | Description              |
|------------------------------|--------------------------|
| http://localhost:3000        | Boutique (UI principale) |
| http://localhost:4000/health | État broker + RAG index  |
| http://localhost:4001        | Product Agent (API only) |
| http://localhost:4002        | Payment Agent (API only) |

### 15.3 Compte démo

Email : demo@pixelium.com · Mot de passe : demo123

### 15.4 Scénarios à montrer à l'encadrant

1. **Parcours classique** : parcourir le catalogue → ajouter au panier → checkout → approuver le mandat → paiement simulé.
2. **Assistant RAG** : « What should I get as a gift under \$30 for a kid who likes trains? » → Get advice → voir recommandations vectorielles.
3. **Achat par IA** : « Buy noise-canceling headphones under \$400 » → Search & review → modale consentement.
4. **Audit** : consulter l'historique des commandes et la chaîne de mandats.
5. **Sécurité** : `npm run test:adversarial` — démontrer le blocage des attaques.

### 15.5 Messages clés pour la soutenance

1. « Le broker est le seul point de contrôle — aucun paiement sans mandat valide. »
2. « L'utilisateur garde le dernier mot : même l'IA ne paie qu'après revue et approbation. »

3. « *Le RAG ne peut pas inventer de produits — il ne recommande que le catalogue indexé.* »
4. « *Les agents Python exécutent ; le broker orchestre ; React présente le consentement.* »

---

Document généré pour le projet Pixelium Consent Commerce — Stage 2026.

Repository : pixelium-consent-commerce · Docs complémentaires : docs/FINAL\_REPORT.md,  
docs/MANDATE\_FORMAT.md, docs/SECURITY\_FINDINGS.md